

Janus4K AI Core Architecture Specification

Version: v0.5-public

Date: 2026-04-26

Status: Public presentation version

Working draft: Janus4k_AS_draft.md

1. Overview

Janus4K is a tile-centric AI Core subsystem organized around a shared `TileReg`. The execution hierarchy is `Block -> Tileop -> uop`. `BCC` performs Block-level dependency resolution, scalar execution, task dispatch, resolve tracking, and in-order retirement. `vector Core`, `Cube`, and `TMA` execute vector computation, matrix computation, and Tile/Memory transfer respectively.

The core architectural rules are:

- `TileReg` is the shared in-core data buffer with `1 MB` total capacity.
- A Vector Tile is fixed at `4 KB`. Cube and TMA may use large Tiles whose sizes are integer multiples of `4 KB`.
- `512 B` is the common local granularity for Vector compute slices, TileReg bank slices, and `Output Buffer` entries.
- `2 KB` is the full-bandwidth TileReg read/write window, formed by 4 logical banks each providing `512 B`.
- `Output Buffer` is the single global structure for result residency, forwarding, writeback arbitration, and reduction reuse.
- Block completion resolve may return out of order. BCC retires Blocks in order.

1.1 Architecture Boundary

This specification defines Janus4K internal Block control, Tile data organization, Vector/Cube/TMA cooperation, TileReg read/write arbitration, and Output Buffer forwarding/writeback semantics. Software stack details, compiler IR, DDR controller internals, NoC protocol, and concrete SRAM macro implementation are outside the scope of this document. External modules must comply with the Block descriptor, Tile granularity, interface handshakes, and completion semantics defined here.

1.2 Visible Behavior

Janus4K exposes the following stable behavior to software and verification environments:

Behavior	Architectural Definition
Block dispatch	BCC dispatches a Block to exactly one target PE after dependencies are satisfied.
Data exchange	Vector, Cube, and TMA exchange Tile data through TileReg.
Vector Tile	Each Vector Tile is fixed at <code>4 KB</code> .
Read/write window	The full-bandwidth TileReg read/write window is fixed at <code>2 KB</code> .

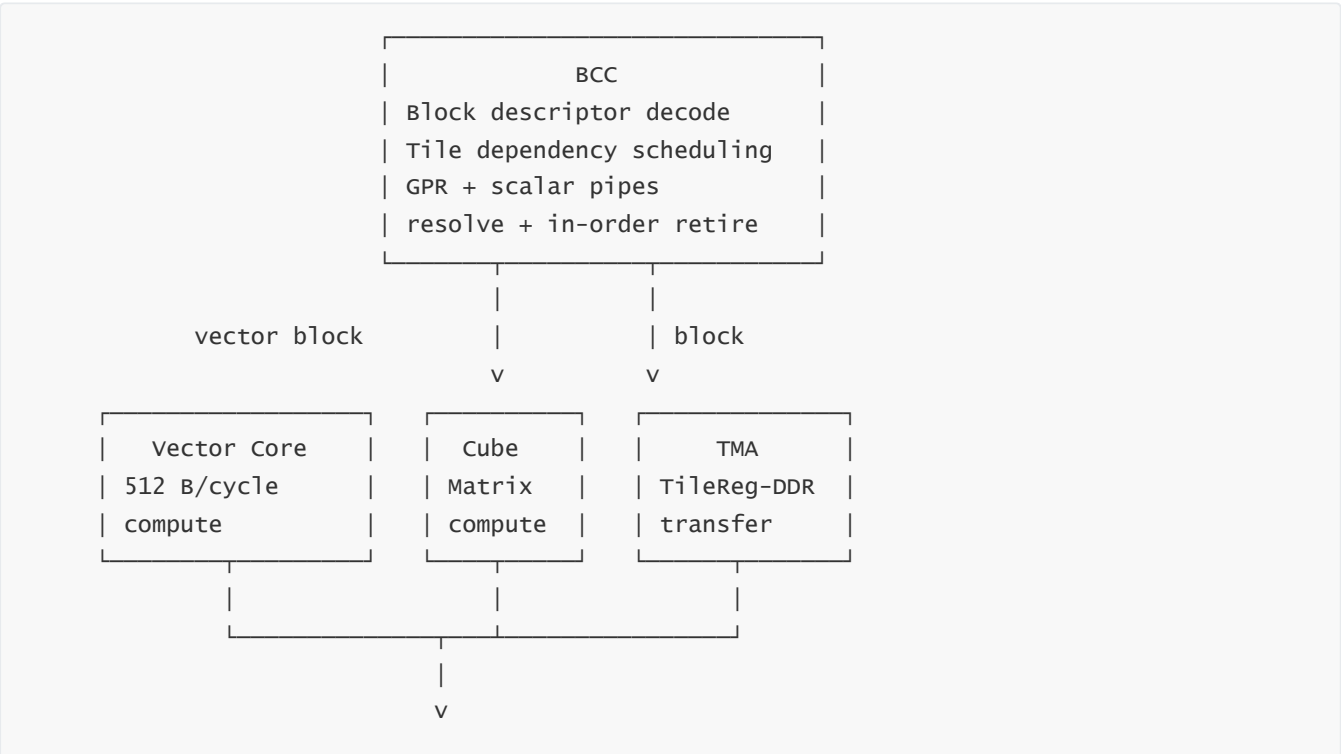
Behavior	Architectural Definition
Result forwarding	Forwardable results in Output Buffer can be consumed before TileReg writeback.
Completion feedback	A PE returns <code>resolve + block_id</code> ; BCC retires Blocks in order.

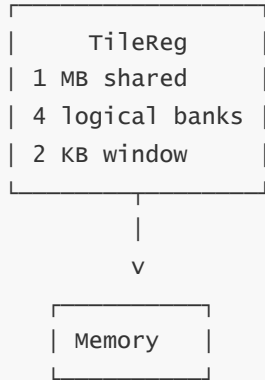
2. Top-Level Architecture

2.1 Top-Level Modules

Module	Responsibility
BCC	Block Control Core. Decodes Block descriptors, resolves Tile dependencies, executes scalar pipes, dispatches tasks, tracks resolve, and retires Blocks in order.
Vector Core	Executes Vector TileOps/uOps with <code>512 B/cycle</code> primary compute granularity.
Cube	Matrix compute unit. Receives TileReg data into L0A/L0B buffers and consumes it through a systolic array.
TMA	Tile Memory Access unit. Moves data bidirectionally between TileReg and DDR/Memory.
TileReg	Shared AI Core data buffer with <code>1 MB</code> total capacity and 4 logical banks.
Memory	External or upper-level memory system.

2.2 Top-Level Diagram





2.3 Data-Centric Organization

`TileReg` is the data convergence point for Vector, Cube, and TMA. Vector Core, Cube, and TMA share `TileReg` ports through read/write arbitration. Data flows through `TileReg`, Global Src Buffer, Output Buffer, and the execution units.

2.4 Control Path and Data Path

Janus4K separates control paths from data paths:

Path	Source	Destination	Payload
Block control	BCC	Vector/Cube/TMA	<code>block_id</code> , opcode, target PE, Tile descriptors, dtype/packing, memory ordering
Resolve control	Vector/Cube/TMA	BCC	<code>resolve_valid</code> , <code>resolve_block_id</code> , completion status
Tile read	Vector/Cube/TMA	TileReg	2 KB read-window request
Tile return	TileReg	Global Src Buffer / Cube / TMA	2 KB read-return data and tag
Result writeback	Output Buffer / Cube / TMA	TileReg	512 B result entry or aggregated writeback data
Memory transfer	TMA	Memory	256 B load/store beats

The control path preserves Block identity. The data path preserves Tile ID, slice/window ID, tag, mask, and dtype semantics, so forwarding, writeback, resolve, and retirement remain correctly associated.

2.5 Single-Target PE Dispatch

Each Block carries a unique `target PE` in `Bstart`. BCC does not duplicate one Block to multiple PEs. Work that requires multiple PEs is represented as multiple Blocks. Each Block independently establishes Tile input/output dependencies and independently returns `resolve + block_id`.

3. Data Model

3.1 Tile

Item	Definition
Vector Tile	Fixed 4 KB
Cube/TMA Tile	Integer multiple of 4 KB
Tile slice	512 B
TileReg access window	2 KB
Tile address organization	A single 4 KB Tile is contiguous in TileReg

A 4 KB Vector Tile contains eight 512 B slices. TileReg maps 512B slice numbers onto 4 logical banks, allowing any PE to read a continuous 2 KB window from a Tile without bank conflict.

The slice and window organization inside a Tile is fixed:

Slice	Byte range	2KB window	Usage
slice0	[0, 512)	window0	Vector compute slice 0
slice1	[512, 1024)	window0	Vector compute slice 1
slice2	[1024, 1536)	window0	Vector compute slice 2
slice3	[1536, 2048)	window0	Vector compute slice 3
slice4	[2048, 2560)	window1	Vector compute slice 4
slice5	[2560, 3072)	window1	Vector compute slice 5
slice6	[3072, 3584)	window1	Vector compute slice 6
slice7	[3584, 4096)	window1	Vector compute slice 7

A full Vector TileOp covers both window0 and window1. Partial Tile operations still use 512 B as the minimum execution slice and 2 KB as the full-bandwidth TileReg access granularity.

3.2 TileReg

TileReg is the shared 1MB data buffer:

Attribute	Value
Total capacity	1 MB
Logical banks	4
Read bandwidth per bank	512 B/cycle

Attribute	Value
Write bandwidth per bank	512 B/cycle
Bank ports	dual-port
Full read window	2 KB/cycle
Full write window	2 KB/cycle

Read and write conflicts are resolved through arbitration. TileReg banks are architectural logical banks. RTL and physical implementation may choose SRAM partitioning, replication, or porting strategies, while preserving the visible bandwidth and conflict-free window semantics.

TileReg requests carry the following logical identity:

Field	Semantics
tile_id	Tile identifier in TileReg
window_id	2 KB window ID, 0 or 1 inside a Vector Tile
slice_base	Base 512 B slice
bytes	Request size, 2048 for a full-bandwidth window
client	Vector compute, Cube, or TMA
tag	Identity for return data, wakeup, writeback, and debug matching

TileReg guarantees that a 2 KB read for the same tile_id + window_id returns without bank conflict after a grant. Writeback may be performed as 512 B entries or as an implementation-level wider write window. The architectural result preserves slice order and byte mask correctness.

3.3 Block Descriptor

Block descriptors are decoded by BCC. Each Block descriptor starts with BStart and uses one or more BCC instructions to describe inputs, outputs, and transfer attributes.

Instruction	Fields	Semantics
BStart	opcode, target PE, dtype/packing, attrs	Marks Block start, operation class, target PE, and data format.
B.IOT	3 src Tiles + 1 dst Tile	Binds input and output Tiles.
TLOAD	memory addr, dst Tile, attrs	TMA load-type Tile transfer instruction.

Constraints:

- 1. A Block is dispatched to exactly one target PE.
- 2. A Block has at most 4 destination Tiles.

3. A Block has at most 2 source Tile descriptor groups.
4. A single `B.IOT` describes `3 src + 1 dst`; multiple `B.IOT` instructions combine to express multi-input or multi-output Blocks.
5. `Bstart` carries dtype/packing. Vector TileOps and uOps use this information to interpret element width and data layout.

Logical `Bstart` fields are:

Field	Semantics
<code>block_id</code>	Unique Block identity
<code>opcode</code>	Block opcode
<code>target_pe</code>	Vector, Cube, or TMA
<code>dtype_pack</code>	Data type and packing
<code>elem_width</code>	Element width
<code>mask_en</code>	Whether a mask Tile is used
<code>reduction_en</code>	Whether the operation is a reduction
<code>memory_order_attr</code>	TMA memory-ordering attribute

Expanded `B.IOT` entries form the Tile dependency table. Each entry contains source Tiles, destination Tiles, read/write attributes, mask attributes, and target PE. BCC uses these entries to establish inter-Block dependencies and releases them when the Block retires.

3.4 Dependency Model

Block dependencies are established through Tile inputs and outputs. Normal compute Blocks do not use flag dependencies and do not create normal memory side-effect dependencies. TMA `TLOAD` and store operations follow memory ordering.

`GPR` is used for scalar value exchange across Blocks. Scalar dependencies are maintained by the BCC scalar pipes and GPR.

4. BCC

4.1 Responsibilities

`BCC` is the Block-level control core of Janus4K. It performs:

1. `Bstart/B.IOT/TLOAD` decoding.
2. Tile input/output dependency tracking.
3. BCC scalar pipe execution.
4. Dispatch of ready Blocks to Vector Core, Cube, or TMA.

5. Reception of `resolve + block_id` from PEs.
6. Out-of-order resolve tracking and in-order Block retirement.

4.2 Scalar Pipe

BCC contains GPR and three scalar pipe classes:

Structure	Function
GPR	General-purpose scalar register file for scalar value exchange across Blocks.
AB pipe	ALU + branch pipe with 2-pick support.
AM pipe	ALU + multicycle pipe with scalar multiply support.
LSU	Scalar load/store unit.

4.3 Resolve and Retirement

After a PE completes a Block, it returns `resolve + block_id` to BCC. Resolve responses may return out of order. BCC records completion state in the retirement structure and retires Blocks in program order. Retirement releases Tile dependencies, GPR dependencies, and state visible to later Blocks. TMA Blocks retire after their memory ordering constraints are satisfied.

5. Vector Core

5.1 Structure

Vector Core is the primary programmable vector execution unit in Janus4K. It receives Vector Blocks from BCC, decomposes TileOps into uOps, and executes them on fixed execution resources.

Structure	Responsibility
vector block / tileop window	Receives Vector Blocks, stores TileOp state, and maintains Tile-level dependency and wakeup state.
TileOp Decoder	Generates a uOp plan from opcode, dtype/packing, element width, mask, and reduction attributes.
operand collector	Looks up Output Buffer first, then generates TileReg read requests for missed sources.
Read Port Arbitration	Performs RR arbitration for TileReg read requests from Vector compute, Cube, and TMA.
Global Src Buffer	Stores 2 KB source data windows returned by TileReg.
uOp Issue Queue	Maintains ready uOps by execution class and issues them to fixed execution units.

Structure	Responsibility
FMLA/FCVT Pipe	Executes FMLA and FCVT/CVT operations.
IALU/PERM/MAC Pipe	Executes IALU, PERM, MAC, compare, and reduction operations.
SFU	Executes EXP/DIV and other special functions at 256 B/cycle.
Output Buffer	Stores results and supports forwarding, 2KB locking, writeback arbitration, and reduction reuse.

5.2 Vector Pipeline Overview

Vector Core pipeline is divided into front-end, source-read, issue, execute, result, and completion sections:

Stage	Name	Main Structure	Input	Output
V0	Block accept	Vector Block	BCC Vector Block	TileOp window entry
V1	TileOp decode	TileOp Decoder	opcode, B.IOT, dtype/packing, mask	uOp plan
V2	Source collect	Operand Collector	src Tile desc, Output Buffer tag	forward hit or read request
V3	Read arbitration	Read Port Arbitration	Vector/Cube/TMA read request	2KB read grant
V4	TileReg read return	TileReg + Global Src Buffer	2KB read window	source window ready
V5	uOp issue	uOp Issue Queue	ready uOp + source window	pipe/SFU issue
V6	Execute	FMLA/FCVT, IALU/PERM/MAC, SFU	512B/256B operand slice	result entry
V7	Result buffer	Output Buffer	result entry	forward hit / writeback request
V8	Completion	Vector Block	TileOp done bitmap	resolve + block_id

Multiple TileOps may reside in the pipeline at the same time. Different TileOps may occupy different stages, and the two 2 KB windows of one TileOp may be pipelined. V3 read arbitration, V5 uOp issue, and V7 Output Buffer insertion independently generate backpressure.

5.3 TileOp to uOp Decomposition

Each Vector TileOp source input is a 4 KB srcTile. A standard elementwise three-source operation is split into two 2 KB windows, and each window is split into four 512 B uOps:

```
TileOp(4KB)
  window0: slice0, slice1, slice2, slice3
  window1: slice4, slice5, slice6, slice7
```

Each three-source TileOp window performs three source reads in a fixed order:

```
windowN:
  read srcA>windowN] -> 2KB
  read srcB>windowN] -> 2KB
  read srcC>windowN] -> 2KB
  issue 4 x 512B uop
```

uOp descriptors contain the following logical fields:

Field	Semantics
block_id	Owning Block
tileop_id	Owning TileOp
uop_id	uOp sequence inside the TileOp
opclass	FMLA/FCVT, IALU/PERM/MAC, or SFU
src_desc[0..2]	Source Tile/window/slice descriptors
mask_tile_desc	Optional mask Tile
dst_desc	Destination Tile/window/slice descriptor
dtype_pack	Data type and packing carried by BStart
elem_width	Element width
window_id	2KB window ID
slice_id	512B slice ID
forwardable	Whether the result may be forwarded after entering Output Buffer
reduction_ctx	Reduction-chain context

Elementwise FMLA/FCVT/IALU/PERM/MAC operations execute at 512 B slice granularity. SFU operations still read data as 2 KB windows, while the execution stage consumes data at 256 B/cycle. Reduction operations generate uOps according to reduction direction, element width, and reduction tree, and intermediate results may remain resident in Output Buffer.

5.4 Source-Read Pipeline

Operand Collector looks up Output Buffer before consuming TileReg read bandwidth:

```
src desc
├─ Output Buffer hit  -> lock 2KB window -> source ready
└─ Output Buffer miss -> enqueue TileReg read request -> wait grant
```

Source-read rules are:

Rule	Definition
Lookup priority	Output Buffer lookup precedes TileReg read request generation.
Hit behavior	A hit source does not consume a TileReg read port; the matching 2KB Output Buffer window is locked.
Miss behavior	A missed source enters Read Port Arbitration.
Grant granularity	Each grant reads one 2 KB window.
Three-source order	A Vector three-source window issues reads in srcA, srcB, srcC order.
Return location	TileReg return data enters Global Src Buffer.
Wakeup	A uOp group wakes up after all source windows are ready.

Global Src Buffer entry granularity is 2 KB. A three-source window may occupy up to three Global Src Buffer entries for srcA, srcB, and srcC. A source satisfied through Output Buffer forwarding does not allocate a Global Src Buffer entry.

5.5 uOp Issue Queue

The uOp Issue Queue is partitioned into three fixed queues:

Queue	Execution Unit	Operations
FMLA/FCVT Queue	FMLA/FCVT Pipe	FMLA, FCVT/CVT
IALU/PERM/MAC Queue	IALU/PERM/MAC Pipe	IALU, PERM, MAC, compare, reduction, TMAX/colmax
SFU Queue	SFU	EXP, DIV, and other special functions

A uOp becomes ready when:

- 1. All source windows are ready.
- 2. The mask Tile is ready, or the uOp does not use a mask.
- 3. The target execution unit can accept the uOp.
- 4. Output Buffer can allocate a result entry or assert backpressure.

5. The previous `reduction_ctx` is ready for reduction uOps.

The three execution resource classes do not share execution units. `opclass` to execution-unit mapping is fixed. The scheduler does not issue SFU uOps to normal Vector pipes and does not issue FMLA/FCVT uOps to the IALU/PERM/MAC Pipe.

5.6 Execute Pipeline

Vector execution includes two 512B main execution paths and one 256B SFU path:

Execution Path	Input Granularity	Output Granularity	Typical Operations	Result Destination
FMLA/FCVT Pipe	512 B/cycle	512 B entry	FMLA, FCVT/CVT	Output Buffer
IALU/PERM/MAC Pipe	512 B/cycle	512 B entry	IALU, PERM, MAC, TMAX/colmax	Output Buffer
SFU	256 B/cycle	Aggregated into 512 B entry	EXP, DIV	Output Buffer

Normal 512B pipes write 512 B entries into Output Buffer. SFU consumes 256 B per cycle, and two consecutive 256B results are combined into one 512 B Output Buffer entry. Execution outputs carry `block_id/tileop_id/uop_id/dst_desc/slice_id/tag` for Output Buffer lookup, writeback, and completion accounting.

5.7 Typical Three-Source TileOp Timing

A standard three-source 4KB TileOp follows this read and compute order:

Step	Action	Data Size	Description
1	window0 srcA read	2KB	RR arbitration
2	window0 srcB read	2KB	RR arbitration
3	window0 srcC read	2KB	RR arbitration
4	window0 compute	4 x 512B	Issue four 512B uOps
5	window1 srcA read	2KB	RR arbitration
6	window1 srcB read	2KB	RR arbitration
7	window1 srcC read	2KB	RR arbitration
8	window1 compute	4 x 512B	Issue four 512B uOps
9	result residency/writeback	8 x 512B	Results enter Output Buffer and write back according to dependency and port state

When a source hits Output Buffer, the corresponding read step is replaced by forwarding and no TileReg read is issued. After all three sources for a window are ready, the four uOps of that window may enter the target issue queue.

5.8 Wakeup and Dependency Handling

Vector Core uses two wakeup classes:

Wakeup	Source	Purpose
External wakeup	TileReg read grant/return	Wakes uOp groups waiting for TileReg data.
Internal wakeup	Output Buffer result ready	Wakes producer-consumer chains or later reduction uOps.

An Output Buffer hit consumer locks the corresponding 2KB window and enters source-ready state. Reduction successors can consume intermediate results directly from Output Buffer and do not wait for intermediate TileReg writeback.

5.9 Vector Backpressure

Every Vector pipeline stage follows valid/ready semantics. If a stage cannot accept new payload, the upstream stage keeps payload stable and preserves tag and data.

Blocking Point	Trigger	Backpressure Direction
TileOp Window full	No available TileOp entry	BCC stops dispatching new Vector Blocks.
Operand Collector busy	Source lookup or read request queue is full	TileOp generation of new uOps pauses.
Read Arbitration pending	RR does not grant	uOp group waits for sources.
Global Src Buffer full	TileReg return has no free entry	TileReg return path applies backpressure and suppresses new read grants.
uOp Queue full	Target opclass queue has no free entry	Operand Collector pauses ready uOp dispatch.
Execute busy	Target execution unit cannot accept	uOp Queue holds ready uOps.
Output Buffer full/locked	No result entry is available or the target window is locked	Execute output applies backpressure and suppresses issue.
TileReg write busy	Write arbitration does not grant	Output Buffer keeps unwritten entries.

6. Output Buffer

`Output Buffer` is a single global structure between execution results and TileReg writeback. It provides:

- 1. Result storage at `512 B` entry granularity.
- 2. Producer-consumer forwarding.
- 3. Cross-pipe forwarding.
- 4. TileReg write-port arbitration.
- 5. Reduction intermediate reuse.
- 6. Result residency before later writeback.

Logical fields of an Output Buffer entry are:

Field	Semantics
<code>valid</code>	Whether the entry is valid
<code>tag</code>	Forwarding/writeback matching tag
<code>tile_id</code>	Destination Tile
<code>window_id</code>	2KB window
<code>slice_id</code>	512B slice
<code>data</code>	512B result
<code>byte_mask</code>	Valid byte mask
<code>forwardable</code>	Whether forwarding is allowed
<code>writeback_required</code>	Whether TileReg writeback is required
<code>locked</code>	Whether the owning 2KB window is locked by a consumer
<code>reduction_ctx</code>	Reduction-chain context

When a TileOp finds its source data in Output Buffer, the corresponding `2 KB` data window is locked. While locked, the window cannot be overwritten or released. It is unlocked after the consumer finishes. Unlocked entries attempt TileReg writeback every cycle, and writeback requests are arbitrated at the TileReg write port.

Reduction intermediate results may stay in Output Buffer across multiple reduction steps. The final reduction result is written back to the destination Tile after writeback conditions are satisfied.

7. TileReg Arbitration and Buffers

7.1 Read Port Arbitration

TileReg read arbitration accepts requests from three clients:

- 1. `Vector compute`
- 2. `Cube`
- 3. `TMA`

The arbitration policy is Round-Robin. Each grant corresponds to one `2 KB` TileReg read window. Requests that do not receive a grant remain pending and continue participating in later RR rounds.

7.2 Global Src Buffer

Global Src Buffer is located on the TileReg return path and buffers `2 KB` read windows. Its entry granularity is `2 KB`, and its depth is 6 entries. Before entering execution units, a `2 KB` data window can be split into four `512 B` slices.

7.3 Write-Port Arbitration

TileReg write conflicts are resolved through write-port arbitration. Output Buffer, Cube, and TMA may all generate write requests. Write arbitration preserves data, tag, tile_id, and slice_id consistency, and it guarantees that forwarding-visible results are not lost when writeback is stalled.

8. Cube

Cube is the matrix compute unit of Janus4K. Cube inputs and outputs are expressed as Tiles, and may use large Tiles composed of multiple `4 KB` Tiles. Cube reads data from TileReg into `L0A/L0B` buffers and consumes it through an internal systolic array. Cube internal consumption granularity is not constrained by the Vector `512 B` slice. Cube returns `resolve + block_id` to BCC after completion.

9. TMA

TMA performs bidirectional data movement between TileReg and DDR/Memory.

Attribute	Definition
Direction	bidirectional load/store
TileReg request limit	<code>2 KB</code>
Memory-side beat	<code>256 B</code>
Read/write ports	separate
Queueing	load/store share scheduling queues
Ordering	TLOAD/store operations maintain memory ordering

TMA load moves memory data into TileReg. TMA store writes TileReg data back to memory. TMA returns `resolve + block_id` to BCC after completion.

10. Vector Execution Flow

1. BCC decodes the Block descriptor and confirms that the target PE is Vector Core.
2. BCC checks dependency readiness through Tile inputs and outputs.
3. BCC dispatches a ready Vector Block to the Vector Block / TileOp Window.
4. TileOp Decoder generates a uOp plan from `Bstart/B.iOT`, dtype/packing, element width, mask, and opclass.
5. Operand Collector performs Output Buffer lookup for each source.
6. Sources that hit Output Buffer lock the corresponding 2KB window and are satisfied through forwarding.
7. Sources that miss Output Buffer generate TileReg read requests to Read Port Arbitration.
8. Read Port Arbitration grants `2 KB` read windows using RR.
9. TileReg return data enters Global Src Buffer.
10. After all sources for a window are ready, four 512B uOps enter the fixed uOp queue.
11. uOp Issue Queue issues uOps to FMLA/FCVT, IALU/PERM/MAC, or SFU according to opclass.
12. Execution units generate `512 B` Output Buffer entries; SFU aggregates two 256B beats into one 512B entry.
13. Output Buffer wakes later consumers by tag or writes back unlocked entries to TileReg.
14. TileOp done state is updated after all windows and slices complete.
15. Vector Core returns `resolve + block_id` to BCC after all TileOps in the Block complete.
16. BCC retires resolved Blocks in order.

11. Interfaces

11.1 BCC to PE

Signal	Direction	Meaning
<code>block_valid</code>	BCC -> PE	Block is valid.
<code>block_ready</code>	PE -> BCC	Target PE can accept the Block.
<code>block_id</code>	BCC -> PE	Block identifier.
<code>block_target</code>	BCC -> PE	Vector/Cube/TMA target.
<code>block_opcode</code>	BCC -> PE	Block operation class.
<code>block_bstart</code>	BCC -> PE	Decoded BStart payload.

Signal	Direction	Meaning
<code>block_iot_vec</code>	BCC -> PE	Tile input/output bindings expanded from B.IOT.
<code>block_attr</code>	BCC -> PE	dtype/packing, element width, reduction, memory ordering, and related attributes.

11.2 PE to BCC

Signal	Direction	Meaning
<code>resolve_valid</code>	PE -> BCC	Block completion resolve is valid.
<code>resolve_ready</code>	BCC -> PE	BCC can accept resolve.
<code>resolve_block_id</code>	PE -> BCC	Completed Block ID.
<code>resolve_status</code>	PE -> BCC	Completion status.

11.3 Operand Collector to Read Port Arbitration

Signal	Direction	Meaning
<code>rd_req_valid</code>	requester -> arb	TileReg read request is valid.
<code>rd_req_ready</code>	arb -> requester	Arbiter can accept the request.
<code>rd_req_client</code>	requester -> arb	Vector compute/Cube/TMA.
<code>rd_req_tile_id</code>	requester -> arb	Tile ID.
<code>rd_req_slice_base</code>	requester -> arb	512B slice or 2KB window base.
<code>rd_req_bytes</code>	requester -> arb	Request size, normally 2 KB.
<code>rd_req_tag</code>	requester -> arb	Tag for return-data matching.

11.4 Execute/SFU to Output Buffer

Signal	Direction	Meaning
<code>ob_valid</code>	exec -> OB	Execution result is valid.
<code>ob_ready</code>	OB -> exec	Output Buffer can accept the result.
<code>ob_tag</code>	exec -> OB	Forwarding/writeback matching tag.
<code>ob_data</code>	exec -> OB	512 B result entry.
<code>ob_forwardable</code>	exec -> OB	Result can be forwarded.
<code>ob_writeback_req</code>	exec -> OB	Result requires TileReg writeback.

Signal	Direction	Meaning
ob_reduction_ctx	exec -> OB	Reduction context.

11.5 TMA

Signal	Direction	Meaning
tma_cmd_valid	BCC -> TMA	TMA command is valid.
tma_cmd_ready	TMA -> BCC	TMA can accept the command.
tma_cmd_is_store	BCC -> TMA	Load/store direction.
tma_tile_id	BCC -> TMA	TileReg Tile ID.
tma_tile_bytes	BCC -> TMA	TileReg request size, maximum 2 KB.
tma_mem_addr	BCC -> TMA	Memory address.
tma_mem_beat_bytes	BCC -> TMA	Memory beat size, fixed at 256 B.
tma_order_tag	BCC -> TMA	Memory ordering tag.

12. Parameter Table

Parameter	Value	Description
TILEREG_BYTES	1 MB	Total TileReg capacity.
TILE_BYTES	4096	Vector Tile size.
TILE_SLICE_BYTES	512	Vector slice / Output Buffer entry size.
TILEREG_BANKS	4	Number of logical TileReg banks.
TILEREG_BANK_BYTES_PER_CYCLE	512	Read/write bandwidth per bank per cycle.
TILEREG_RW_BYTES_PER_CYCLE	2048	Full-bandwidth TileReg read/write window.
GLOBAL_SRCBUF_ENTRY_BYTES	2048	Global Src Buffer entry size.
GLOBAL_SRCBUF_ENTRIES	6	Global Src Buffer depth.
OUTPUT_BUFFER_ENTRY_BYTES	512	Output Buffer entry size.
MAX_DST_TILES_PER_BLOCK	4	Maximum destination Tiles per Block.
MAX_SRC_TILE_DESC_GROUPS_PER_BLOCK	2	Maximum source Tile descriptor groups per Block.
B_IOT_SRC_TILES	3	Number of source Tiles in one B.IOT.

Parameter	Value	Description
B_IOT_DST_TILES	1	Number of destination Tiles in one B.IOT.
NUM_EXEC_CLASSES	3	FMLA/FCVT, IALU/PERM/MAC, and SFU.
SFU_BYTES_PER_CYCLE	256	SFU EXP/DIV bandwidth.
TMA_REQ_BYTES	2048	Maximum TMA TileReg request size.
TMA_MEM_BEAT_BYTES	256	TMA memory-side beat size.
READ_ARB_POLICY	RR	TileReg read arbitration policy.

13. Verification Coverage

Verification covers the following architectural behavior:

1. BCC BStart/B.IOT/TLOAD decoding and single-target PE dispatch.
2. BCC scalar pipe behavior, GPR scalar value exchange across Blocks, and AB/AM/LSU behavior.
3. Vector 4 KB three-source TileOp execution through two 2 KB read windows and eight 512 B compute slices.
4. Output Buffer hits do not consume TileReg read bandwidth.
5. Output Buffer 2 KB lock/unlock behavior and writeback arbitration for unlocked entries.
6. Reduction intermediate residency in Output Buffer.
7. RR read arbitration across Vector/Cube/TMA requests.
8. Conflict-free 2 KB continuous read windows over TileReg logical banks.
9. SFU EXP/DIV 256 B/cycle behavior and mask Tile input.
10. TMA 2 KB TileReg requests, 256 B memory beats, bidirectional load/store, and memory ordering.
11. Cube LOA/LOB buffering and large-Tile input/output.
12. Out-of-order PE resolve return and in-order BCC retirement.
13. Vector pipeline backpressure at TileOp Window, Operand Collector, Read Arbitration, Global Src Buffer, uOp Queue, Execute, Output Buffer, and TileReg writeback.

14. Glossary

Term	Meaning
BCC	Block Control Core
Block	Task granularity visible to BCC
TileOp	Tile-level Vector Core operation
uOp	Fine-grained operation issued to an execution unit

Term	Meaning
TileReg	Shared Janus4K Tile register/buffer
Output Buffer	Global result residency, forwarding, and writeback arbitration structure
Global Src Buffer	2 KB entry buffer after TileReg read return
BStart	Block descriptor start instruction
B.IOT	Tile input/output binding instruction
TLOAD	TMA load-type Tile transfer instruction
GPR	BCC scalar general-purpose register file
SFU	Special Function Unit
Resolve	Block completion feedback from PE to BCC